

VIBE CODING

想法 × 设计 × 开源

用 AI 开发，用思路设计，用结果验证。



```
PS C:\ClubRecruit> whoami
>> 会用 GitHub, 备好 AI 编程环境
PS C:\ClubRecruit> Get-Content .\tracks.txt
>> 开源贡献 / PR | 自主项目 / 插件
PS C:\ClubRecruit> echo $focus
>> 开发思路 · 设计取舍 · 验证与采纳
```

Vibe Coding 是主线，开源是实践场景

Vibe Coding：开发方式

opencode

新会话

工作区

iflab

PPT制作方案与工具推荐 1小时

zcode

get_market

auto_pa

town-core

blackhole

deepseek-harness

blackhole

glm_test

cot-test

ds_vllite_mrcr

hmnxt

some_ddl

SERVERS

PPT制作方案与工具推荐

对话 概述

Session log 5

1. 写了一个脚本 build_ppt.py，用 python-pptx 库。

2. 在 13.33×7.5 英寸 (16:9) 的画布上，每一页都是硬编码坐标去画 [矩形/文本框/线条] ——比如 rect(Inches(0.7), Inches(2.0), Inches(5.85), ...) 这种绝对位置。

3. 配色是深绿色 + 绿色高亮 (模仿原 PPT 的终端风)，每个文字 run 单独设字号、颜色、字体 (中文微软雅黑 / 等宽 Consolas)。

4. 生成后用 PowerPoint COM 渲染成 PNG，逐页看有没有溢出/错位，再改坐标重跑。

为什么排版会有问题 (根本原因)

这种「用代码把每个元素摆到固定坐标」的方式有三个天生短板：

- 文字一长就溢出/换行难看 (你之前看到的句号“被挤到单独一行就是典型)；
- 不同电脑/字体渲染结果可能不同，绝对坐标在字体不一致时容易对不齐；
- 很难达到原版那种精雕细琢——原 PPT 的排版是设计出来的，我这版是「手工算坐标凑出来」的，所以空档、间距难免别扭。

所以你说的「排版还是有点问题」我完全理解，是方法局限。

接下来你想怎么处理? (选一个方向)

1. 你告诉我具体哪里不对劲 (哪页? 字大小/太大? 间距、对齐、空档、元素重叠?)，我直接在脚本里改那一处；

2. 我用 PPT 当原型，保留它排版已经很精致的模板/版式，我负责替换文字内容 (用 COM 或者啥改

你负责思路与设计，AI 参与实现。

开源：协作与验证

<> Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

hm_driver_rs Public

main 1 Branch 0 Tags

Go to file Add file

shenjackyuanjie feat(driver): 支持多前台任务与整树等待 0b019be · 3 hours ago 44 C

assets	feat(agent): 更新 Hypium 26.0.0.400 Agent	3 we
src	feat(driver): 支持多前台任务与整树等待	3 hc
tests	feat(input): 添加 Hypium 26 指关节操作	3 we
.gitignore	feat(driver): 实现原生 HarmonyOS 驱动层	las
Cargo.lock	feat(driver): 支持多前台任务与整树等待	3 hc
Cargo.toml	feat(driver): 支持多前台任务与整树等待	3 hc
README.md	feat(input): 添加 Hypium 26 指关节操作	3 we
THIRD_PARTY_NOTICES.md	feat(agent): 更新 Hypium 26.0.0.400 Agent	3 we

自研项目与开源 PR，都欢迎。

两个子赛道，同一套评价标准

子赛道 A

开源贡献 / PR

- › 修问题、补功能、改文档。
- › 理解需求，用 AI 协作设计与修改。
- › 提交 PR，跟进 review、验证与采纳。

例：为 DSH 修问题、补功能，提交 PR。

子赛道 B

自主项目 / 插件

- › 从零开发，或迭代自己的已有项目。
- › 可复用开源组件、方案与 AI 工具。
- › 讲清设计取舍，展示验证与使用反馈。

例：开发 DSH 插件；自选课题需提前沟通。

已有项目：只评提交窗口内的新增工作。

你最终要交什么？

两个子赛道，均需提交以下四项。

- | | | |
|----|---|--------|
| 01 | 项目 / PR 与复现说明
公开 GitHub 仓库 / PR · README (运行、版本、改动) · 关键 commit | [必需] |
| 02 | 开发思路与设计说明
需求 · 方案比较 · 设计取舍 · 开源复用 | [必需] |
| 03 | AI 协作关键记录
关键提示词 / Agent 记录 · 人的判断、纠错与迭代 · 敏感信息脱敏 | [必需] |
| 04 | 验证与采纳结果
测试 · PR 状态 / 用户反馈 · 无反馈也如实说明 | [必需] |

演示材料：建议附截图或短视频。

开源贡献：从发现问题到获得反馈

① 确认问题

理解并复现

② 阅读与设计

比较方案

③ AI 协作开发

检查误改

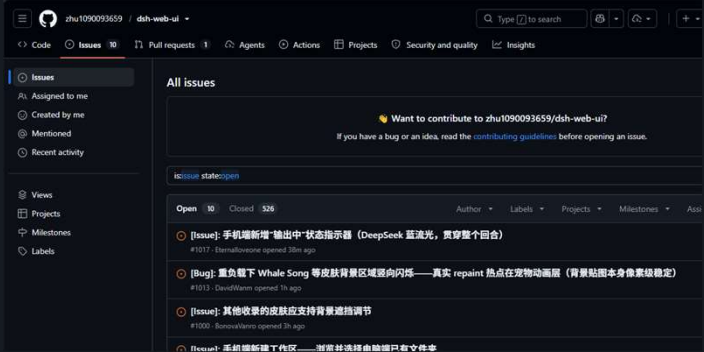
④ 测试与提交

测试 + PR

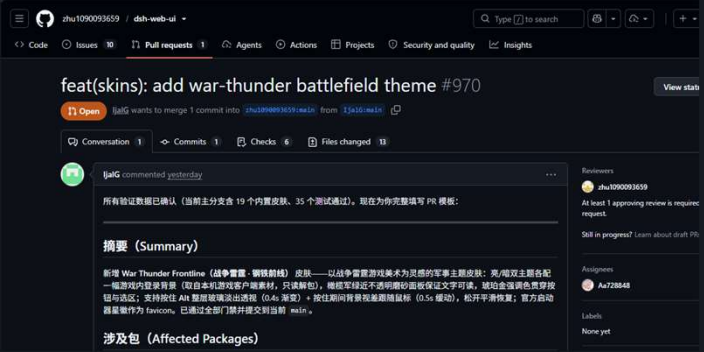
⑤ 跟进反馈

review + 采纳

Issue：理解需求



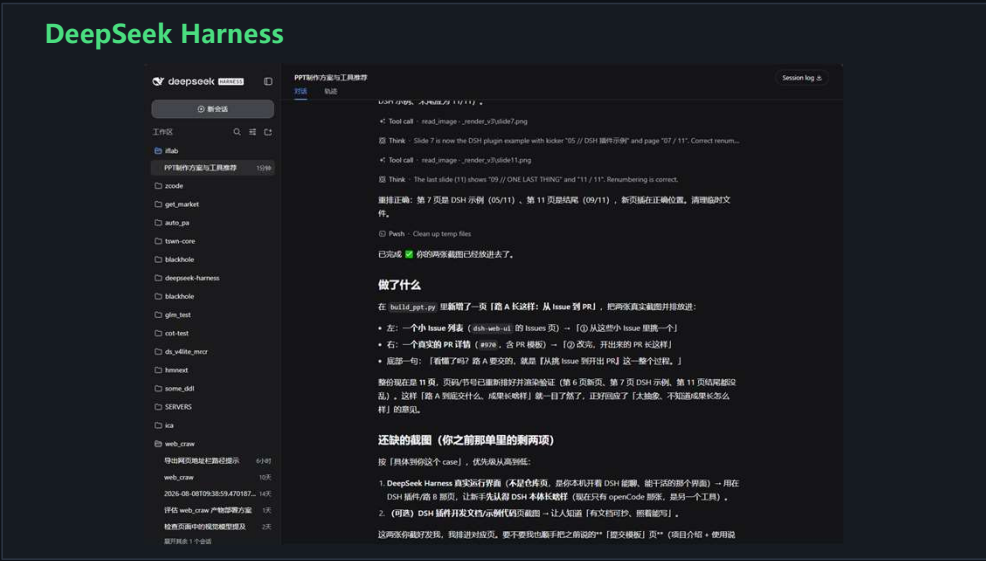
PR：提交改动与验证



采纳看证据：测试 / review / PR 状态；未合并 ≠ 零分，需说明原因。

自主开发：从一个 DSH 插件开始

DeepSeek Harness (DSH) 插件，也可作为独立参赛作品。



换工具字标

参考项目：[shenjackyuanjie/dsh-sfw](https://github.com/shenjackyuanjie/dsh-sfw)



改标签页名称

定制欢迎区

同样做 DSH，怎样体现思路与设计？

两个选题示意

子赛道 A // 向上游提 PR

让插件配置报错更易定位

思路：复现配置报错，确认需求。

设计：提示位置、信息与回退。

协作：AI 定位补测，人检查误改。

结果：测试、review 与 PR 状态。

追问：为什么值得上游采纳？

子赛道 B // 自主开发插件

做一个可配置的界面插件

思路：明确用户与定制需求。

设计：配置化方案与插件边界。

协作：AI 分步实现，人验证兼容。

结果：他人试用、反馈与改进。

追问：他人能否真正用起来？

插件交付：DSH / API 版本 · 运行环境 · 已知兼容性

怎么打分：思路、设计与结果优先

开发思路与选题价值 需求 · 拆解 · 取舍		30
设计性与解决方案 架构 · 交互 · 边界 · 复用		25
验证、采纳与实际价值 测试 / review / 试用反馈		20
AI 协作与迭代能力 上下文 · 纠错 · 验证		15
完成度与交付规范 核心成果 · README · 提交记录		10

说在最后

AI 参与开发，
你负责思路、设计与判断。
自研一个项目，或贡献一份有价值的 PR。

```
PS C:\ClubRecruit> Write-Output "See you in the repo"
PS C:\ClubRecruit> git push origin future
```

期待你的第一个作品，或第一份被认真讨论的 PR。

